UNIVERSITÉ LAVAL

TP2 : topologie et modules

Avant de débuter

```
library(tidyverse)
library(igraph)   # APRÈS le tidyverse
```

Deux parties : **A**, le réseau de Game of Thrones du TP1 ; **B**, un réseau de co-expression construit à partir d'`airway`. Les algorithmes de communautés et de disposition sont **aléatoires** : `set.seed(1)` juste avant chaque appel.

Les données

Fichier	Contenu
`/home/public/reseaux/got_edges.csv`, `got_sex.csv`	les fichiers du TP1
`/home/public/reseaux/airway_coexpr.tsv`	les **150 gènes les plus variables** d'`airway` (expression `vst` de DESeq2) : `gene` (Ensembl), `symbole`, `chromosome`, `statut` (`induit`, `réprimé` ou `non DE` sous dexaméthasone), puis 8 colonnes d'échantillons nommées `donneur_untrt` / `donneur_trt`

```
got_aretes <- read_csv("/home/public/reseaux/got_edges.csv")
got_sexe   <- read_csv("/home/public/reseaux/got_sex.csv")
got <- graph_from_data_frame(got_aretes, directed = FALSE, vertices = got_sexe)
coexpr <- read_tsv("/home/public/reseaux/airway_coexpr.tsv")
```

Rappel : les fonctions

```
degree(g); strength(g)                          # degré ; force (somme des poids)
betweenness(g, weights = NA)                    # intermédiarité, sans les poids
distances(g, "A", "B", weights = NA)            # longueur du plus court chemin
shortest_paths(g, "A", "B", weights = NA)$vpath[[1]]   # le chemin lui-même
diameter(g, weights = NA); get_diameter(g, weights = NA)
```

```r
make_ego_graph(g, order = 1, nodes = "A")[[1]] # le nœud A et ses voisins
clique_num(g); largest_cliques(g)
igraph::components(g)                          # $no, $csize, $membership
set.seed(1); comm <- cluster_louvain(g)        # communautés
length(comm); sizes(comm); membership(comm); modularity(comm)
plot(comm, g)
graph_from_adjacency_matrix(adj, mode = "undirected")   # matrice 0/1 → graphe
```

À votre tour

Partie A : Game of Thrones

0. Reconstruisez `got` (non dirigé, avec le sexe). Vérifiez l'en-tête : `U`, 127 nœuds, 550 liens.
1. Tracez l'histogramme des degrés (`ggplot()`, `geom_histogram(binwidth = 2)`). Combien de personnages ont au plus 3 liens ? Donnez les 5 personnages de plus fort **degré** et de plus forte **force**. Le classement est-il le même ?
2. Calculez l'intermédiarité avec `weights = NA`, puis **sans** préciser `weights`. Donnez les 5 premiers de chaque. Qui entre, qui sort ? Lequel des deux classements a un sens ici, et pourquoi ?
3. Quelle est la distance entre Jon et Daenerys (`weights = NA`) ? Par qui passe le plus court chemin ? Quel est le diamètre du réseau, et entre quels personnages (`get_diameter()`) ?
4. Construisez le réseau **ego** de Jon (`make_ego_graph()`, ordre 1). Combien de nœuds et de liens ? Pourquoi le nombre de nœuds vaut-il `degree(got, "Jon") + 1` ? Dessinez-le avec les étiquettes.
5. Quelle est la taille de la plus grande clique (`clique_num()`) ? Combien y a-t-il de cliques de cette taille ? Quels personnages sont dans **toutes** (`Reduce(intersect, ...)` sur les noms) ?
6. Détectez les communautés avec `cluster_louvain()` (après `set.seed(1)`). Combien de communautés, de quelles tailles ? Quelle modularité ? Dans quelle communauté sont Ned et Daenerys ? Qui est avec Daenerys ? Dessinez avec `plot(comm, got, vertex.label = NA, vertex.size = 4)`.

Partie B : un réseau de co-expression

7. Importez `airway_coexpr.tsv`. Construisez une matrice `m` : les **symboles** en noms de lignes (`column_to_rownames()`), les 8 colonnes d'échantillons seulement. Vérifiez sa dimension. Calculez la matrice des corrélations `cor(t(m))`, appelée `r` : pourquoi `t()` ? Mettez à 1 les corrélations > 0.8, à 0 les autres et la diagonale, puis construisez un graphe **non dirigé** `g_co`. Ajoutez l'attribut `statut`.
8. Combien de nœuds, de liens ? Combien de gènes **isolés** ? Combien de composantes, de quelles tailles (`igraph::components()`) ?
9. Quels sont les 6 gènes de plus fort degré (les **hubs**) ? Quel est leur statut ?
10. Détectez les modules avec `cluster_louvain()` (`set.seed(1)`). Croisez les modules avec le statut, puis avec le chromosome (`table()`). Que remarquez-vous ?
11. Un module ne contient que des gènes **non DE**. Lesquels ? Sur quel chromosome ? Regardez l'expression de *RPS4Y1* et de *XIST* dans les 8 échantillons. Que représente ce module ? Est-ce un effet de la dexaméthasone ?

Et si… ?

Pour chaque question, **prédisez d'abord la réponse sans exécuter**, puis vérifiez dans R.

1. Et si on liait les gènes selon `abs(r) > 0.8` ? Combien de liens ? Les modules sont-ils encore purs (`table()` module × statut) ?
2. Et si le seuil était 0.9 au lieu de 0.8 ? Combien de liens, d'isolés, de composantes ?
3. Et si on construisait le même réseau (seuil 0.8) sur une matrice **aléatoire** de 150 × 8 (`set.seed(1)`, `matrix(rnorm(150 * 8), nrow = 150)`) ? Combien de liens ? Louvain trouve-t-il des modules ? Quelle modularité ?
4. Et si on oubliait `weights = NA` dans `distances()` et `shortest_paths()` entre Jon et Daenerys ? Par qui passe le « plus court » chemin ?
5. Et si on lançait Louvain sur `got` avec les graines 1 à 5 ? Le nombre de communautés est-il toujours le même ? Ned et Daenerys sont-ils parfois ensemble ?
6. Et si on chargeait `library(DESeq2)` **après** igraph, puis appelait `components(g_co)` ? Lisez les messages de chargement.

Pour aller plus loin

Interpréter un module. Récupérez les identifiants Ensembl (`gene`) du plus grand module de gènes induits et lancez une ORA comme au module d'enrichissement. Quel univers choisir ? (Pas les 150 gènes : ils ont été choisis parce qu'ils varient.)

```
library(clusterProfiler)
library(org.Hs.eg.db)
res <- read_tsv("/home/public/enrichissement/airway_deseq2.tsv")
ora_module <- enrichGO(gene = ..., universe = res$gene[!is.na(res$padj)], OrgDb = org.Hs.
                       keyType = "ENSEMBL", ont = "BP", readable = TRUE)
```

Attention : ces packages masquent `components()` (Et si 6).

Une vraie distance. Transformez les poids de `got` en distances, `1 / E(got)$weight` : deux personnages qui interagissent beaucoup sont **proches**. Recalculez le chemin Jon → Daenerys et le top 5 de l'intermédiarité avec `weights = 1 / E(got)$weight`. Comparez avec `weights = NA` et avec le défaut.